FINANCIAL RISK PLATFORM

WS4 | Backend Development Plan

Risk Operations, Scenarios, Reconciliation & Reporting

Workstream objective: Turn calculation services into a governed operational workflow with scenario comparison, traceability and evidence.

Product intent: A broad, lightweight, extensible platform foundation. Initial data volumes and module depth are limited deliberately; architecture, control points and expansion interfaces are not treated as throwaway demo code.

> **Synchronization rule: This document is paired with the opposite-side document for the same workstream. Feature IDs, phase IDs, endpoint contracts, statuses and acceptance criteria must remain aligned.**

Shared architecture: Layer 1 Universal Platform Foundation → Layer 2 Configurable Financial Domain → Layer 3 Financial Calculation Modules → Layer 4 Intelligence and Experience. Data governance and security cross-cut all layers.

1. Scope and Feature Coverage

Feature ID	Feature	Scope / implementation intent
RO-01	Workflow orchestration	Create, run, monitor and review calculation jobs.
RO-02	Scenario management	Baseline and proposed scenario overlays without mutating approved configuration.
RO-03	Calculation lineage	Trace metrics to inputs, transformations, rules and engine versions.
RO-04	Benchmark and reconciliation	Compare output against approved reference values and tolerances.
RO-05	Reports and exports	Generate run summaries, variance reports and controlled exports.
RO-06	Exception and approval handling	Queue failures, assign review state and require approval for consequential actions.
RO-07	Run history and reproducibility	Re-run with pinned versions or explain why a run cannot be reproduced.

2. Phase-Wise Delivery

P0 | Operational lifecycle

Define job/run/scenario states, owners, error states and approval points.

P1 | Run orchestration

Create jobs, status endpoints, retry boundaries and run history.

P2 | Scenario comparison

Run baseline/proposed scenarios against identical input snapshots and compare deltas.

P3 | Lineage and evidence

Persist lineage links, input/rule versions, warnings and test evidence.

P4 | Reconciliation and reporting

Compare with reference values and export a reviewable report.

P5 | Operational controls

Approval queues, exception review, permissions and reproducibility checks.

3. Implementation Responsibilities

Backend responsibilities

• Implement job/run state machine with idempotency where practical, correlation IDs and defined failure/retry behaviour.

• Create isolated scenario overlays and preserve immutable baseline configurations.

• Persist calculation lineage and version references for all results.

• Implement benchmark comparison with configured tolerances and explicit reference-data provenance.

• Generate export/report payloads from stored run results; apply authorization and audit export events.

• Implement approval/review state transitions and reproducibility checks.

4. Paired API / Feature Contract Matrix

Endpoint names are proposed contracts for alignment, not a claim that the APIs already exist. Confirm payload schemas in the shared OpenAPI contract before implementation.

Shared feature pair	Capability	Proposed API contract
FE-RO-01 / BE-RO-01	Run centre	POST /api/v1/jobs; GET /api/v1/jobs/{job_id}; GET /api/v1/runs
FE-RO-02 / BE-RO-02	Scenario compare	POST /api/v1/scenarios; POST /api/v1/scenarios/{scenario_id}/run; GET /api/v1/scenarios/{scenario_id}/comparison
FE-RO-03 / BE-RO-03	Lineage	GET /api/v1/calculation-runs/{run_id}/lineage
FE-RO-04 / BE-RO-04	Benchmark/reconciliation	POST /api/v1/reconciliations; GET /api/v1/reconciliations/{id}
FE-RO-05 / BE-RO-05	Reports/review	POST /api/v1/reports; GET /api/v1/reports/{report_id}; POST /api/v1/reviews/{id}/decision

5. Cross-Cutting Rules

• The canonical model and source mapping are versioned separately. Unknown source attributes are preserved or quarantined by explicit policy, not silently discarded.

• Every calculation run must bind dataset snapshot, schema/mapping version, rule-set version and calculation-engine version.

• Server-side authorization is authoritative. The frontend may hide unavailable actions but does not enforce security.

• No AI-generated code is executed. AI output is validated against schemas and policy, and consequential actions require authorized approval.

• All errors use a structured response with code, message, details and correlation_id. Sensitive payloads and secrets must not be logged.

• Prototype financial modules use synthetic/approved data, documented assumptions and test fixtures; do not claim regulatory certification or Moody's parity.

• The platform is designed for breadth, but production-grade depth is not implied for every financial domain. ALM/IRRBB, collateral, capital, live market data and regulatory feeds require their own specifications and validation.

6. Definition of Done

• Every feature ID in this document has a matching feature ID in its paired frontend/backend document and in the shared feature register.

• Each endpoint has request/response schemas, status enums, authorization expectations and error examples in OpenAPI.

• Frontend components use actual API responses or clearly labelled mocks matching the contract.

• Backend tests cover validation, permissions, version binding and domain logic appropriate to this workstream.

• The cross-workstream integration test passes for the features included in the release.

Change-control rule: Any change to endpoint, field name, enum, versioning behaviour or acceptance criteria must be reflected in both paired documents and the shared contract in the same change.